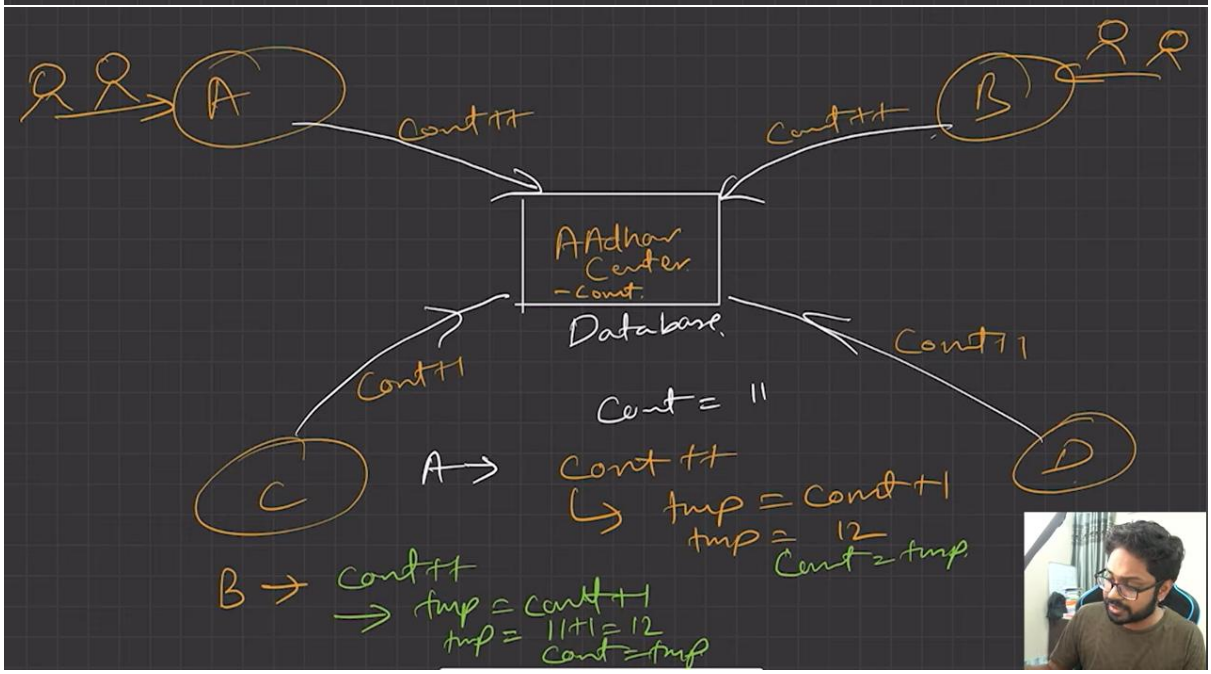
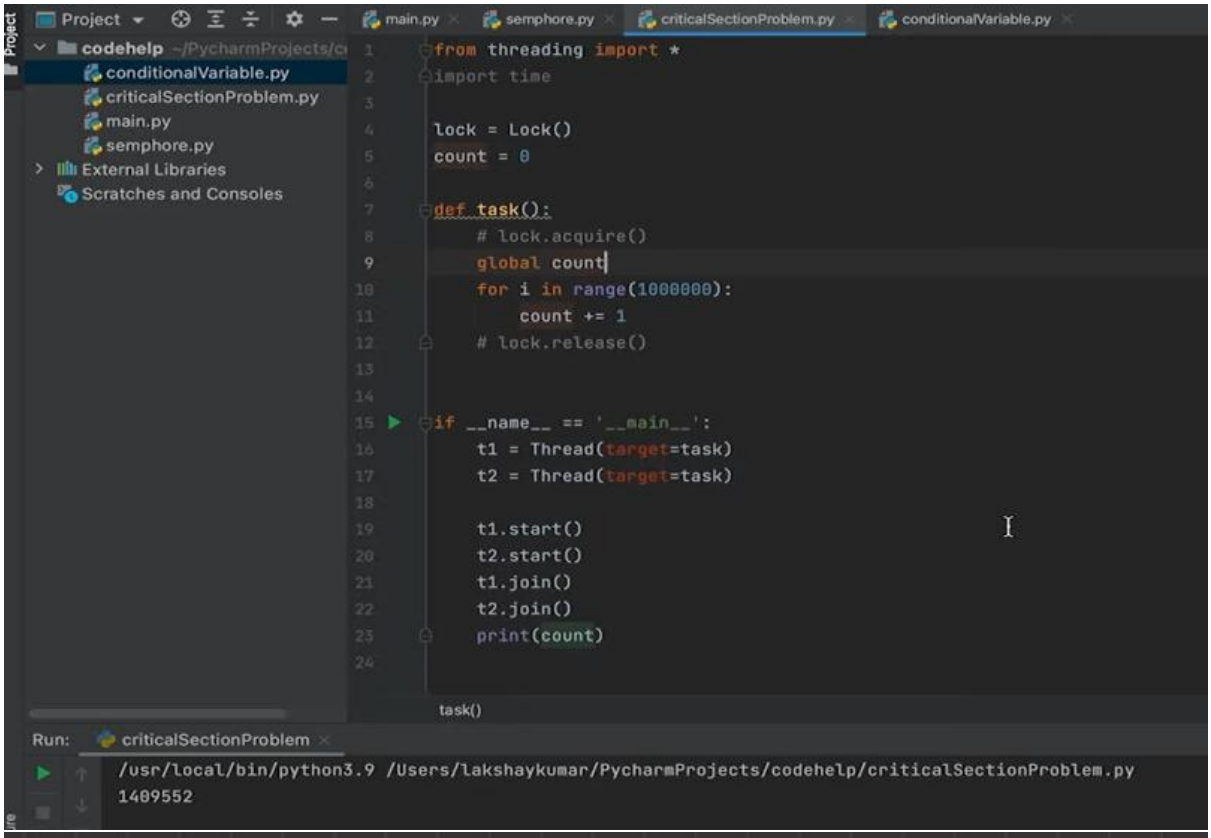
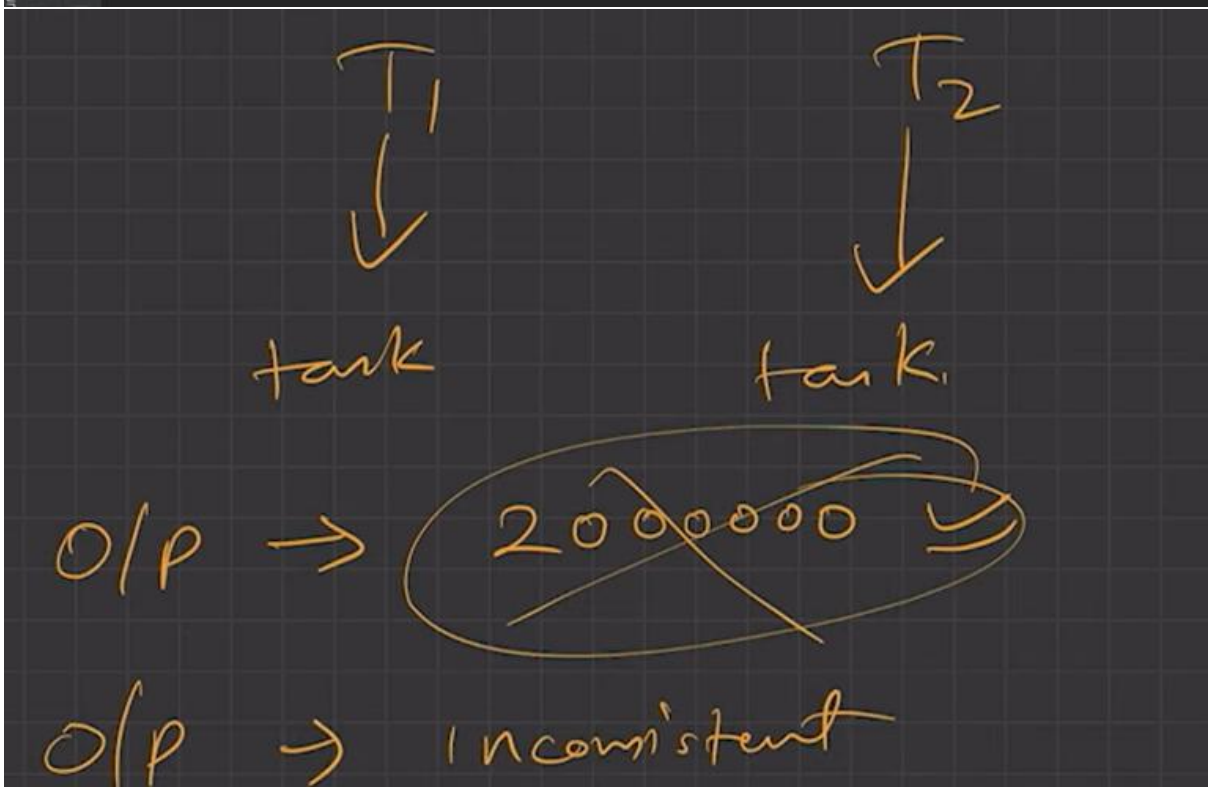
Lec-16

Count ++ → count = count + 1

tmp = count + 1
count = tmp.




Project
codehelp ~/PycharmProjects/c
conditionalVariable.py
criticalSectionProblem.py
main.py
semphore.py
External Libraries
Scratches and Consoles
main.py
semphore.py
criticalSectionProblem.py
conditionalVariable.py
from threading import *
import time
lock = Lock()
count = 0
def task():
lock.acquire()
global count
for i in range(1000000):
count += 1
lock.release()
if __name__ == '__main__':
t1 = Thread(target=task)
t2 = Thread(target=task)
t1.start()
t2.start()
t1.join()
t2.join()
print(count)
task()
Run:
criticalSectionProblem
/usr/local/bin/python3.9 /Users/lakshaykumar/PycharmProjects/codehelp/criticalSectionProblem.py
1409552


T1
T2
task
task
o/p → 2000000
o/p → Inconsistent

Critical section problem

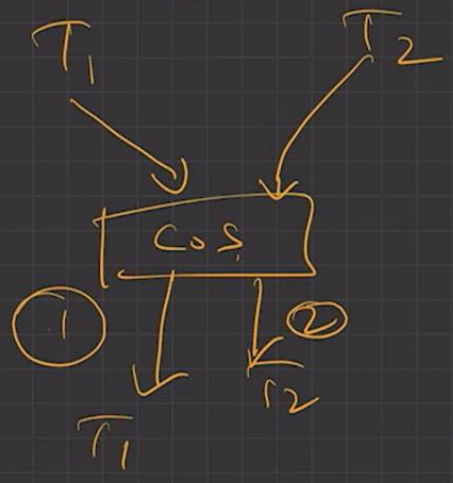
→ Solution of Race cond.

① count++ → tmp = count+1 ✓
count = tmp ✓

Atomic → ✓

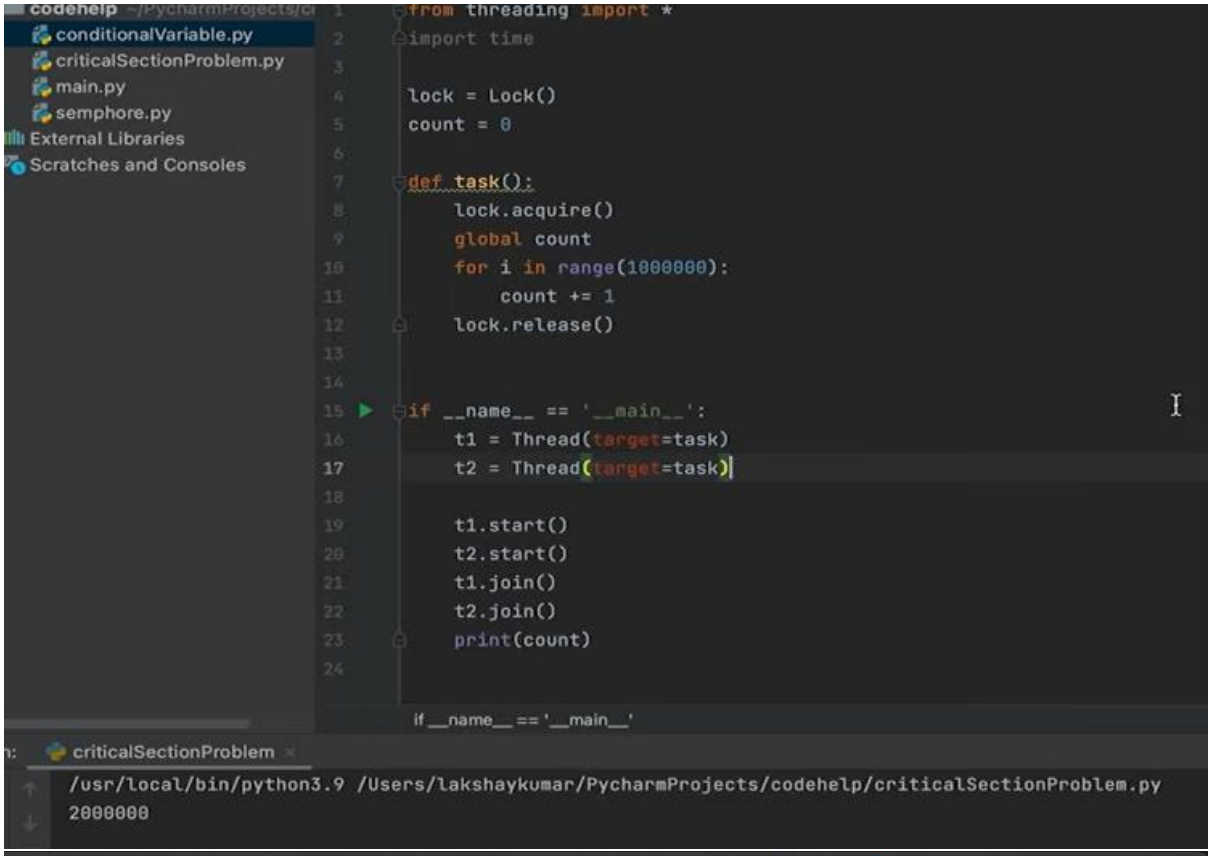
C++ → atomic<int> →

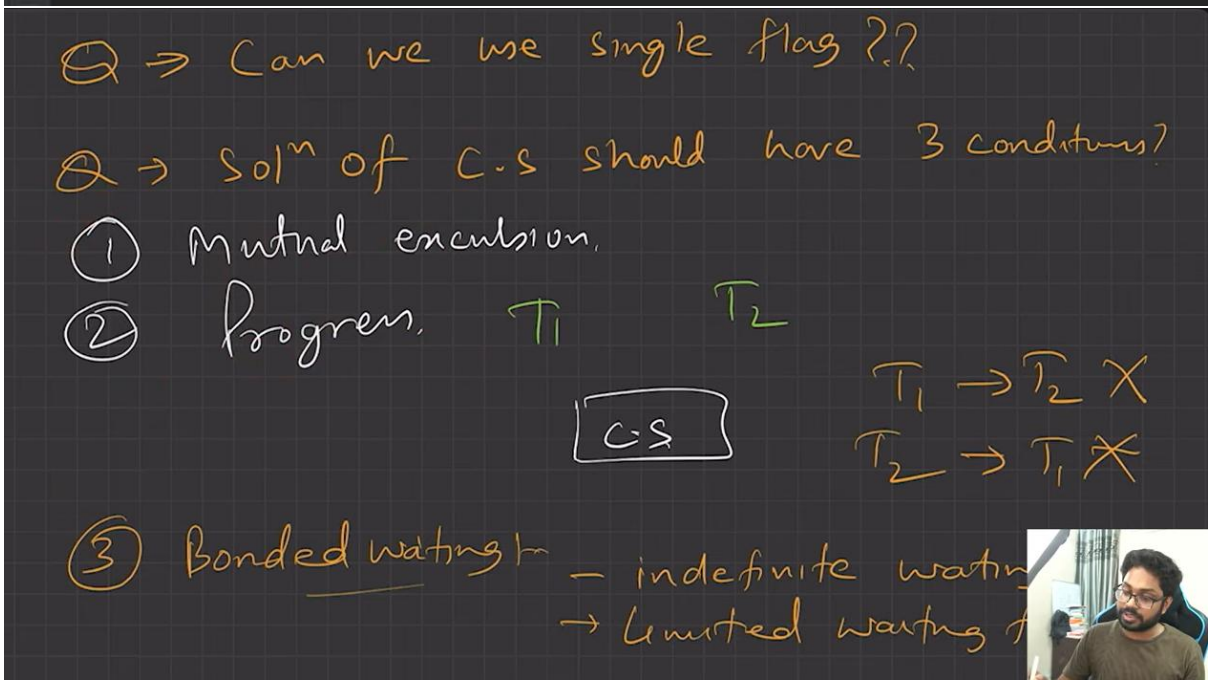
② Mutual Exclusion.



T1 → T2
OR
T2 → T1

Locks → Mutex


codehelp
conditionalVariable.py
criticalSectionProblem.py
main.py
semphore.py
External Libraries
Scratches and Consoles
from threading import *
import time
lock = Lock()
count = 0
def task():
lock.acquire()
global count
for i in range(1000000):
count += 1
lock.release()
if __name__ == '__main__':
t1 = Thread(target=task)
t2 = Thread(target=task)
t1.start()
t2.start()
t1.join()
t2.join()
print(count)
if __name__ == '__main__'
criticalSectionProblem
/usr/local/bin/python3.9 /Users/lakshaykumar/PycharmProjects/codehelp/criticalSectionProblem.py
2000000


Q → Can we use single flag ??
Q → Soln of C.S should have 3 conditions?
① Mutual exculsion.
② Progress. T1 T2
CS
T1 → T2 X
T2 → T1 X
③ Bonded waiting :- indefinite wait
→ Limited waiting

Q Single flag ? → turn = flag = 0/1

T_1

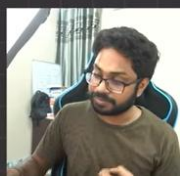
```
while (1)
{
  while (turn != 0);
    C.S
  turn = 1
    R.S
}
```

T_2

```
while (1)
{ while (turn != 1);
  C.S
  Turn = 0
  R.S
}
```

① → turn = 0 → T_1 → T_2

turn = 1 → T_2 → T_1



② Progress — fixed order

=



* Peterson's solution — flag [2] →
— turn

flag [2] → indicate if a thread is ready to enter the C.S, flag [i] = true implies that P_i is ready

turn → indicates whose turn is to enter the C.S

T_1

```
while(1)
{
  flag[0] = T
  turn = 1
  while (turn == 1 && flag[1] == T);
  C.S
  flag[0] = F
}
```

T_2

```
while (1)
{
  flag[1] = T
  turn = 0
  while (turn == 0 && flag[0] == T);
  C.S
  flag[1] = F
}
```

flag [ ][ ]

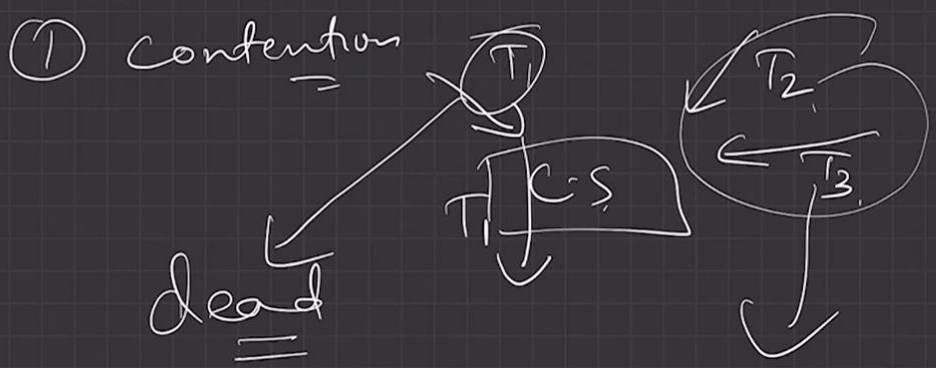
turn :

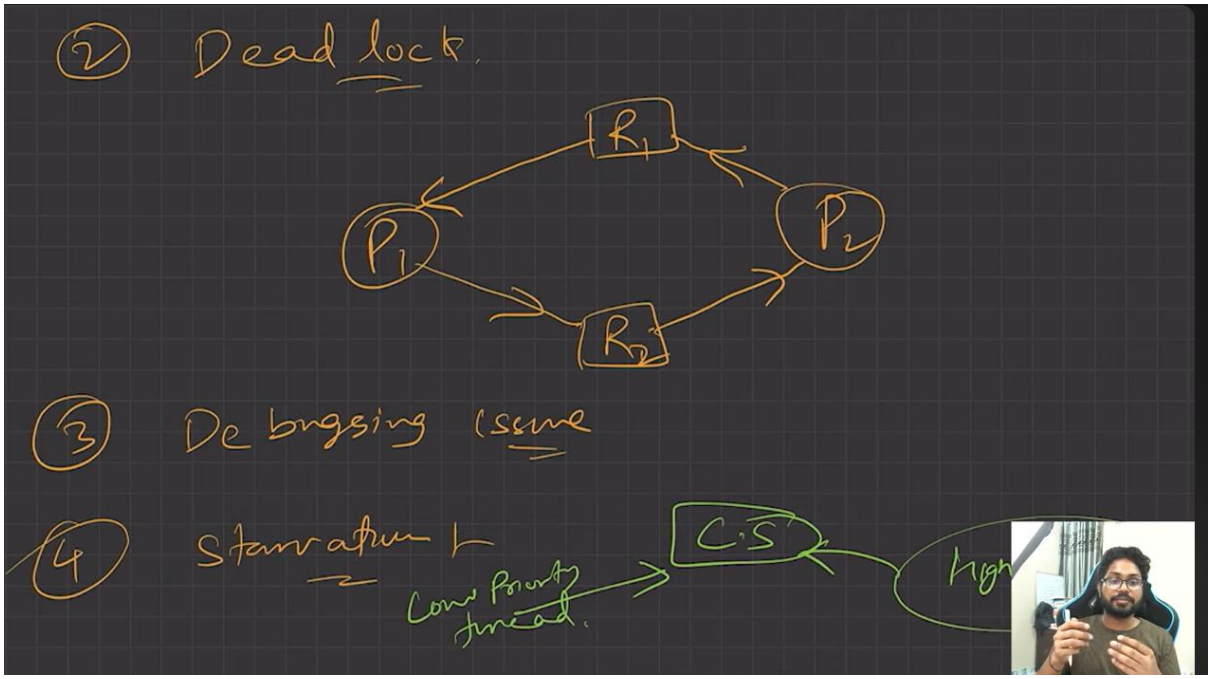
Poolgen



Locks disadvantages

① contention




② Deadlock.
R1
P1
P2
R2
③ Debugging issue
④ Starvation
C.S
Low Priority thread.
High